



Protocol Audit Report

Version 1.0

Cyfrin.io

September 26, 2026

Protocol Audit Report

Ali Talal

March 7, 2023

Prepared by: Ali Talal Lead Auditors: - Ali Talal

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Likelihood & Impact:
 - * [H-2] `PasswordStore::setPassword` has no Access controls, meaning a non-owner could change the password
 - Likelihood & Impact:
 - Informational
 - * [I-1] The `PasswordStore::getPassword` natspec indicates a parameter that doesn't exist, causing the natspec to be incorrect
 - Likelihood & Impact:

Protocol Summary

PasswordStore is a protocol dedicated to storage and retrieval of a user's passwords. The protocol is designed to be used by a single user, and is not designed to be used by multiple users. Only the owner should be able to set and access this password.

Disclaimer

Ali Talal team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

The findings described in this document correspon the following commit hash:

Scope

./src/ #- PasswordStore.sol

```
1
2  ## Roles
3
4  - Owner: The user who can set the password and read the password.
5  - Outsides: No one else should be able to set or read the password.
6
7  # Executive Summary
8
9  - Add some notes about how the audit went, types of things you found,
    etc.*
10 - We spent X hours with Z auditors using Y tools. etc*
11
12 ## Issues found
13
14 | Severtity | Number of issues found |
15 | ----- | ----- |
16 | High      | 2 |
17 | Medium    | 0 |
18 | Low       | 0 |
19 | Info      | 1 |
20 | Total     | 3 |
21
22 # Findings
23
24 ## High
25
26 ### [H-1] Storing the password on-chain makes it visable to anyone, and
    no longer private.
27
28
29 **Decription:** All data stored on-chain is visible to anyone, and can
    be read directly
30 from the blockchain. The `PasswordStore::s_password` variable is
    intended to be a private
31 variable ans only accessed throught the `PasswordStore::getPassword`
    function, which is intended to be only called by the owner of the
    contract.
32
33 We shown one such methos of Reading any data off chain below.
34
35 **Impact:** Anyone can read the private password, severly breaking the
    functionality of the protocol.
36
37 **Proof of Concept:** (Proof of Code)
38
39
```

```
40
41 The below thes case shows how anyone can read the password directly
    from the blockchain.
42
43 1. Create a locally running chain
44 ``` bash
45 make anvil
```

2. Deploy de contract to the chain

```
1 make deploy
```

3. Run the storage tool We use 1 because that´s the storage slot of `s_password` in the contract

```
1 cast storage <ADDRESS_HERE> 1 --rpc-url http://127.0.0.1:8545
```

You will get an output that look likes this: `0x6d7950617373776f726400000000000000000000000000000000000000000000`

You can then parse that hex to a string with:

```
1 cast parse-bytes32-string 0
    x6d7950617373776f72640000000000000000000000000000000000000000000014
```

And get an output of:

```
1 myPassword
```

Recommended Mitigation: Due to this, the overall architecture of the contract should be rethought. One could encrypt the password off-chain, and then store the encrypted password on-chain. This would require the user to remember another password off-chain to decrypt the password. However, you'd also likely want to remove the view function as you wouldn't want the user to accidentally send a transaction with the password that decrypts your password.

Likelihood & Impact:

- Impact: HIGH
- Likelihood: HIGH
- Severity: HIGH

[H-2] PasswordStore::setPassword has no Access controls, meaning a non-owner could change the password

Decription: The `PasswordStore::setPassword` function is set to be an `external` function, however, the natspec of the function and overall purpose of the Smart contract is that This function allows only the owner to set a **new** password

```
1 function setPassword(string memory newPassword) external {
2   @>    // @audit - There are no access controls
3     s_password = newPassword;
4     emit SetNetPassword();
5 }
```

Impact: Anyone can set/change the password of the contract, severely breaking the contract intended functionality.

Proof of Concept: Add the following to the `PasswordStore.t.sol` test file.

Code

```
1     function test_anyone_can_set_password(address randomAddress) public
2     {
3         vm.assume(randomAddress != owner);
4         vm.prank(randomAddress);
5         string memory expectedPassword = "myNewPassword";
6         passwordStore.setPassword(expectedPassword);
7
8         vm.prank(owner);
9         string memory actualPassword = passwordStore.getPassword();
10        assertEq(actualPassword, expectedPassword);
11    }
```

Recomended Mitigation: Add an Access control conditional to the `setPassword` function.

```
1 if(msg.sender != s_owner){
2     revert PassworStore_NotOwner();
3 }
```

Likelihood & Impact:

- Impact: HIGH
- Likelihood: HIGH
- Severity: HIGH

Informational

[I-1] The `PasswordStore::getPassword` natspec indicates a parameter that doesn't exist, causing the natspec to be incorrect

Description:

```
1      /*
2      * @notice This allows only the owner to retrieve the password.
3      * @param newPassword The new password to set.
4      */
5      function getPassword() external view returns (string memory) {}
```

The `PasswordStore::getPassword` function signature is `getPassword()` while the natspec says it should be `getPassword(string)`.

Impact: The natspec is incorrect

Proof of Concept: nothing

Recommended Mitigation: Remove the incorrect natspec line

```
1 -    *@param newPassword The new password to set.
```

Likelihood & Impact:

- Impact: HIGH
- Likelihood: NONE
- Severity: INFORMATIONAL/GAS/NON-CRITS